



Version control with Git & Github



Today's Lesson

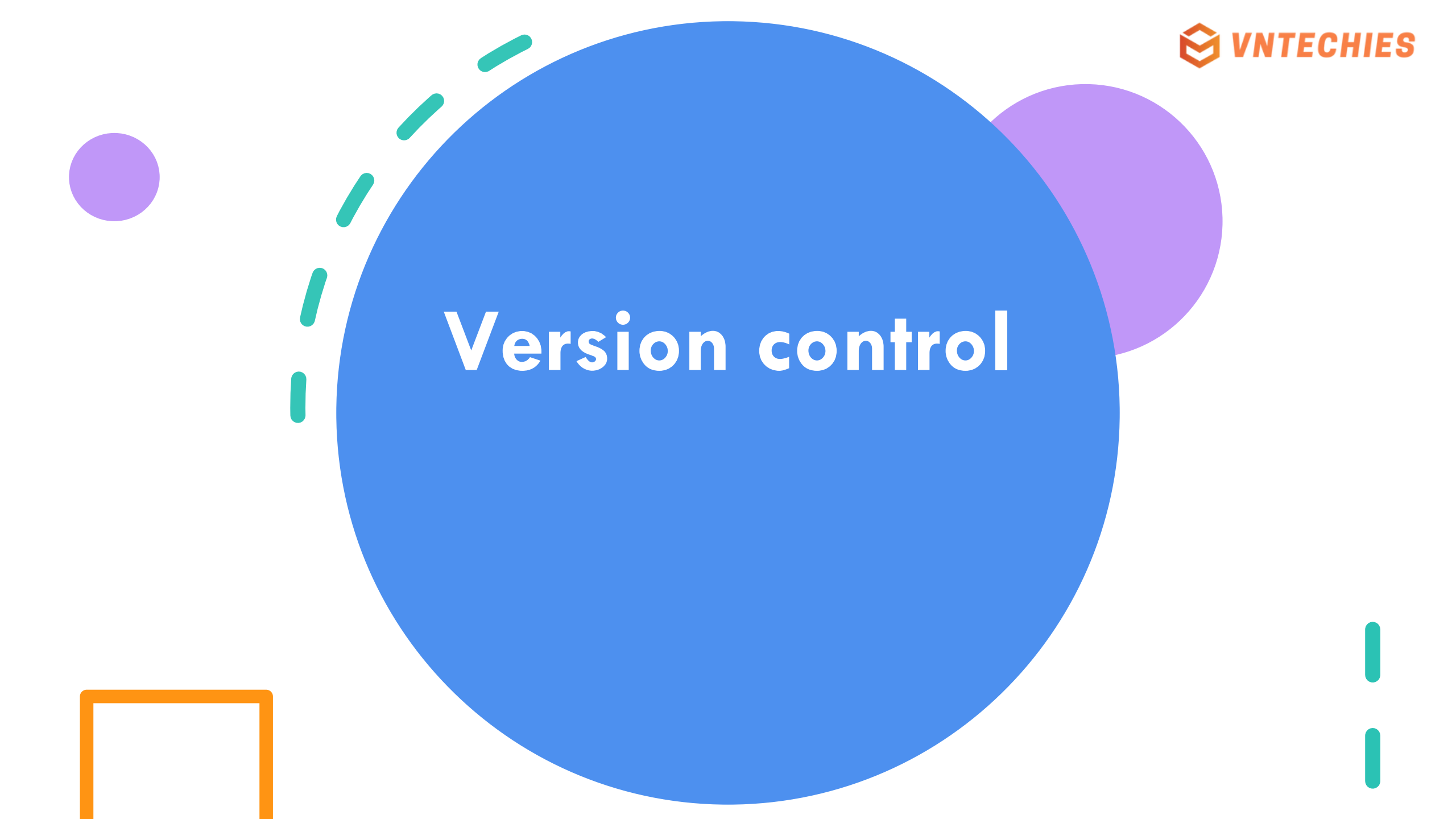
Version control concepts and Git architecture

Git commands and workflows

Implement branching strategies

Collaborate using GitHub

Merge conflicts and advanced Git operations



Version control

What is Version Control

- **Version Control** is a system that helps you **track changes to files over time**, usually code, but also documents, configs, or designs.
- It allows multiple people to work on projects **without losing data, without overwriting each other's work**, and lets you go **back to an earlier version** if something breaks.
- Think version control like a **Magic Box**, people can **throw items in, pull it out**. Every time things come in, the Magic Box tracking it and **store every version of the item** and we can get **any version of the item anytime**

"Version Control = A time machine + collaboration tool for your projects."



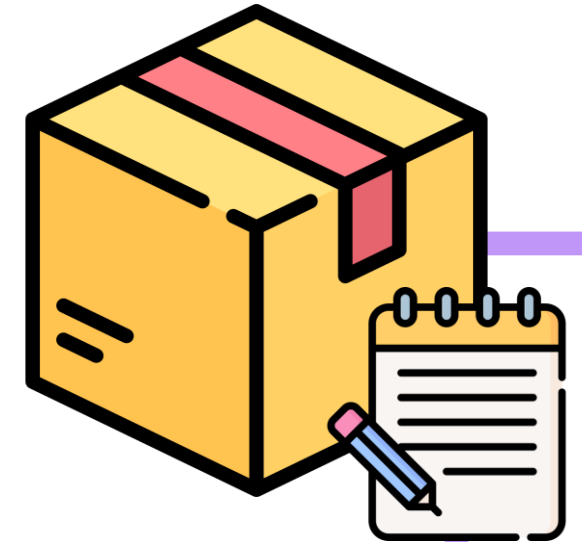
Why Version Control

Without Version Control

- - 📁 `project\_final.zip`
- - 📁 `project\_final\_v2.zip`
- - 📁 `project\_final REALLY\_FINAL.zip`
- - 📁 `project\_final\_use\_this\_one.zip`
- - 🤪 Which version is current?
- - 😬 What changed between versions?
- - 💥 Lost work when files get corrupted

With Version Control

- - 📊 Complete history of all changes
- - 👥 Easy collaboration with team members
- - 🎯 Track who changed what and when
- - ⏮ Rollback to any previous version
- - 🌿 Parallel development with branches
- - 🗄 Backup and redundancy



Git & GitHub

Git

- Git is a **tool** (a distributed version control system).
- It runs **locally on your computer**.
- It **tracks changes** in files (commits), manages version history, and lets you branch & merge.
- You can use Git **without the internet**.

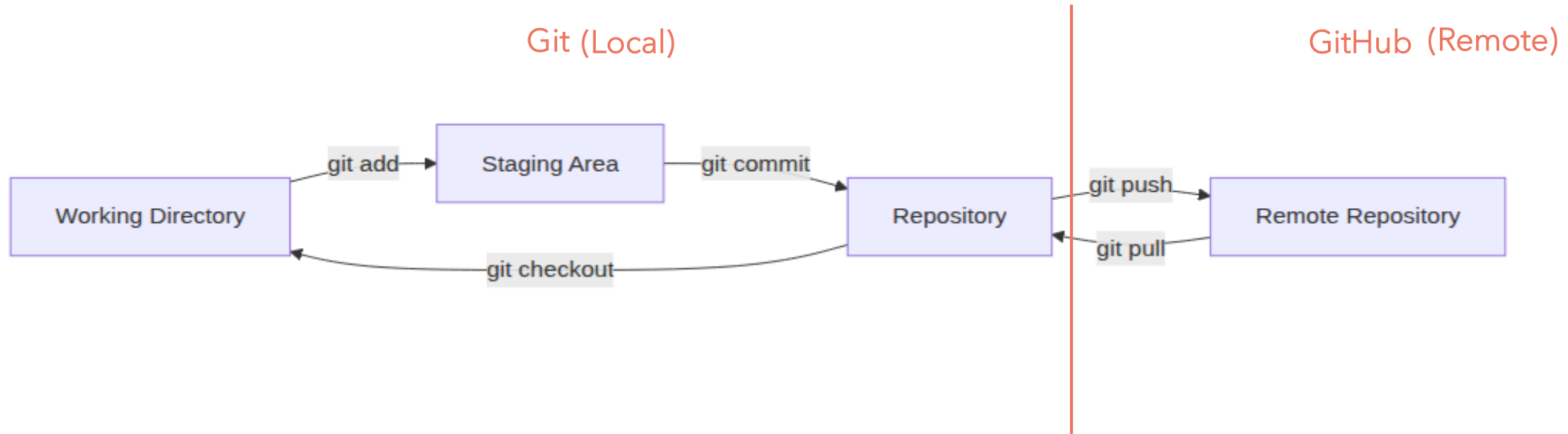
Git = "I save versions of my work."

GitHub = "I share my work with others in the cloud."

GitHub

- GitHub is a **cloud platform** (a website) that hosts Git repositories.
- It provides collaboration features:
 - Remote repositories (store your projects online).
 - Pull Requests (code review before merging).
 - Issues (bug tracking, task management).
 - Actions (automation, CI/CD).

Git Architecture



Working Directory

- Your actual files
- Where you make changes
- Untracked changes

Staging Area

- Prepared changes
- Ready for commit
- Selective commits

Repository

- Committed history
- Permanent record
- Shareable versions

Git History



- **Git history** is the recorded timeline of all the changes (commits) that have been made in a Git repository.
- Every time you **commit** in Git, you're saving a **snapshot** of your project. Git stores:
 - **Who** made the change,
 - **When** it was made,
 - **What** was changed,
 - A unique ID (**hash**) for the commit,
 - A **commit message** describing the change.
- All these commits together form the **Git history**.



Essential Git Commands

Repository Setup

```
# Initialize new repository
git init

# Clone existing repository
git clone <url>

# Configure user
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
```

Basic Workflow

```
# Check status
git status

# Add files to staging
git add <file>
git add .

# Commit changes
git commit -m "Commit message"
```

Remote Operations

```
# Add remote
git remote add origin <url>

# Push changes
git push origin main

# Pull changes
git pull origin main
```

Viewing History

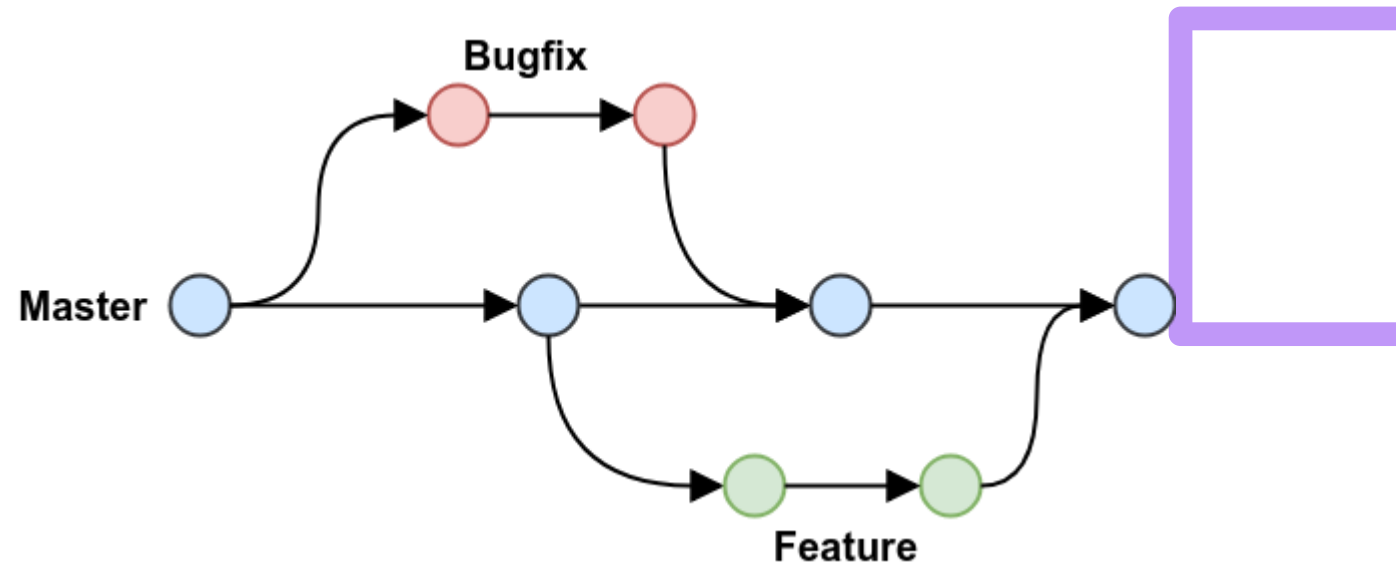
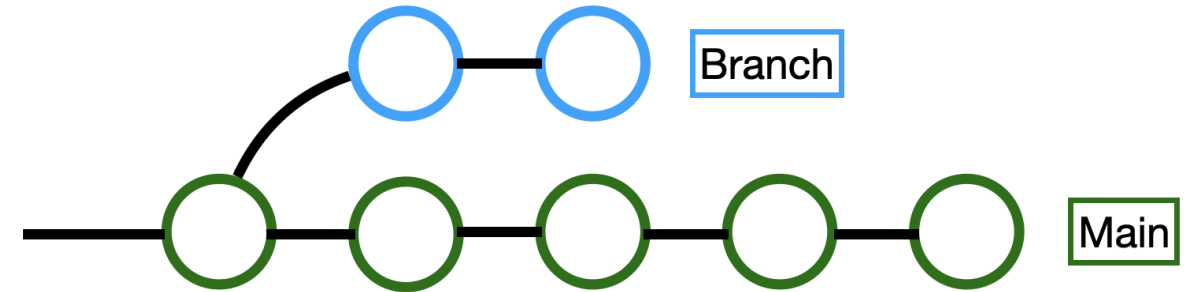
```
# View commit history
git log
git log --oneline
git log --graph

# View changes
git diff
git diff --staged

# Show specific commit
git show <commit-hash>
```

Git Branch

- A Git branch is like a **separate line of development** in your project. It allows you to **work on new features, fix bugs, or experiment** without touching the main (stable) code.
- By default, when you create a Git repository, it starts with one branch – usually called **main** (older repositories may use **master**).



Git Branch

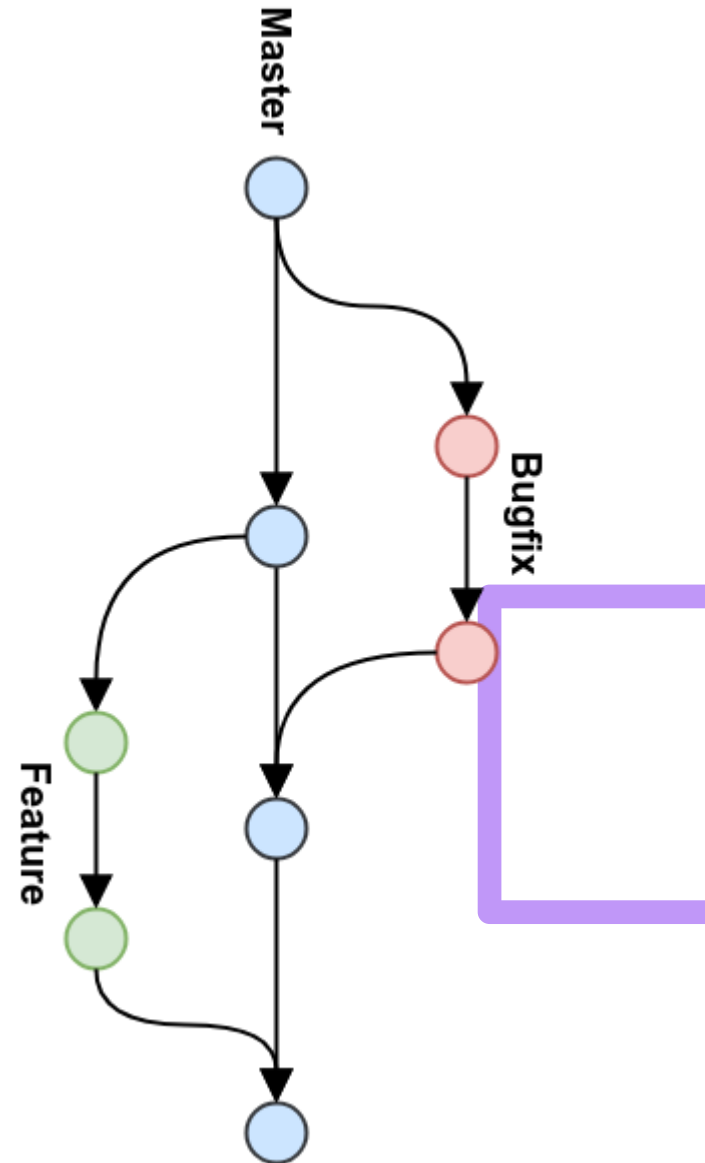
Command

```
# List branches
git branch

# Create new branch
git branch feature

# Switch to branch
git checkout feature
git switch feature

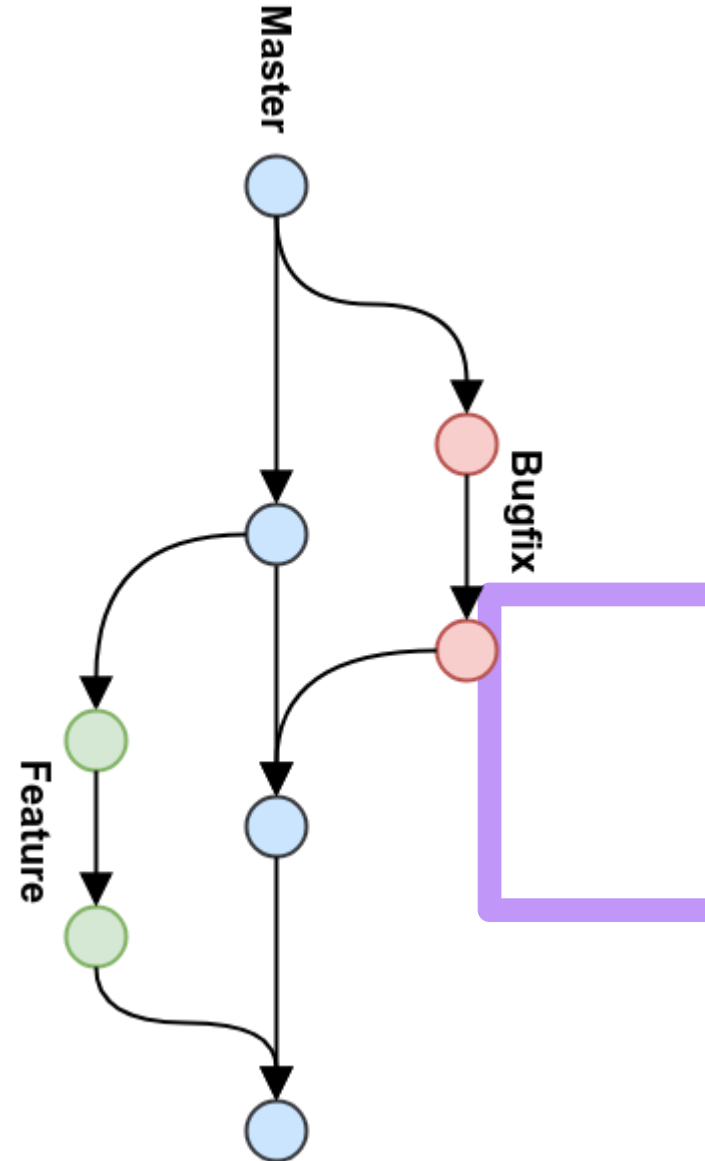
# Create and switch
git checkout -b feature
```



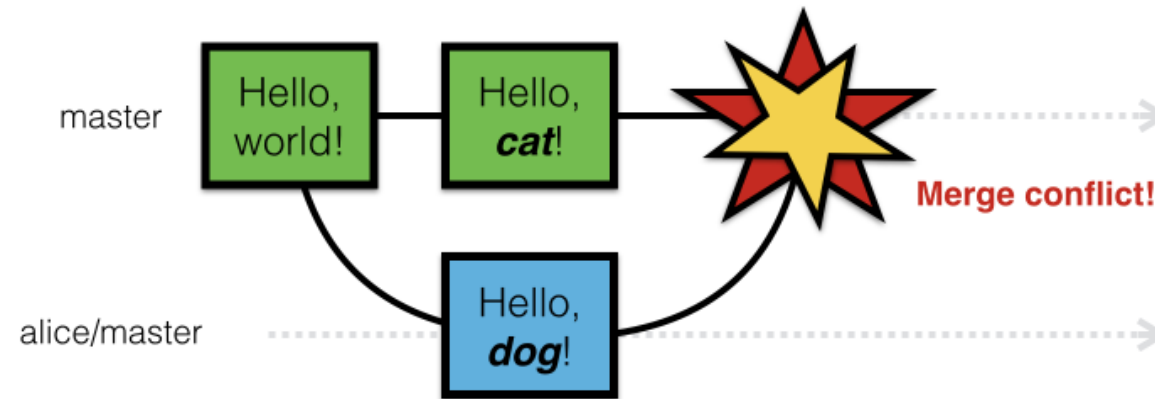
Git Merge

Command

```
#switch to main branch  
git checkout main  
# merge code from feature branch to main  
git merge feature
```



Git Conflict



- A **Git conflict** happens when **Git cannot automatically merge changes** from two different commits or branches because the *same parts of a file* were modified differently.
- Git tries to merge changes automatically, but if it detects overlapping edits in the same section, it stops and asks you to **resolve the conflict manually**.

Resolve Conflict

Step 1: Open the file

- You'll see the conflict markers (<<<<<<, =====, >>>>>>).
- <<<<<< HEAD → changes from your current branch.
- ===== → divider.
- >>>>>> branch-name → changes from the branch being merged.

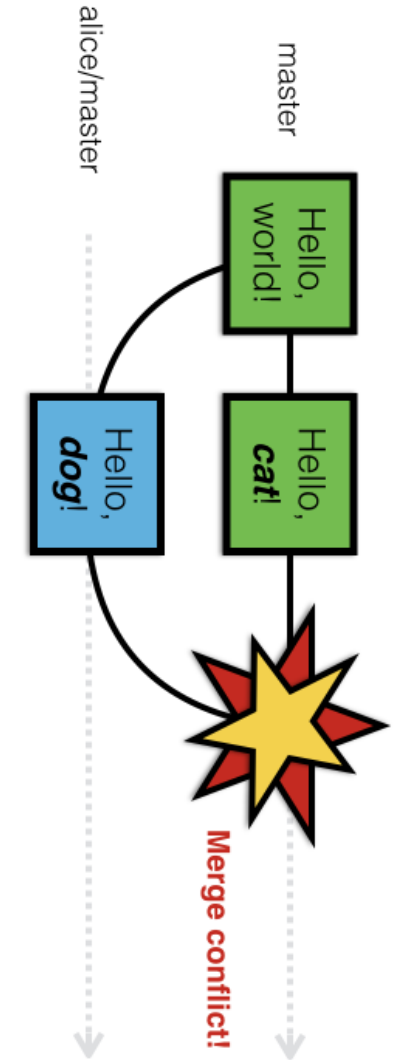
Step 2: Decide the correct version

Edit the file manually to what you want. Example:

- Hello, dog!

Step 3: Mark conflict as resolved

- git add hello.txt
- git commit -m "Resolved merge conflict in hello.txt"





Git

Advance Command

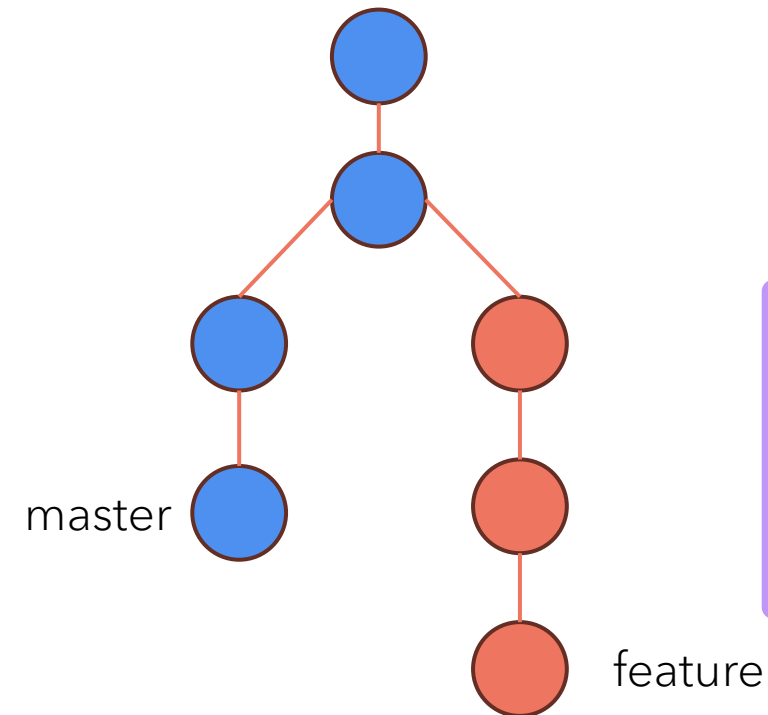
Git Rebase

- In Git, **rebasing** means moving or combining a sequence of commits onto a new base commit. In simpler words: it **re-applies** commits from one branch on top of another branch.
- So instead of a messy "branch merge commit," rebasing "replays" your changes as if you started from the latest commit of another branch.



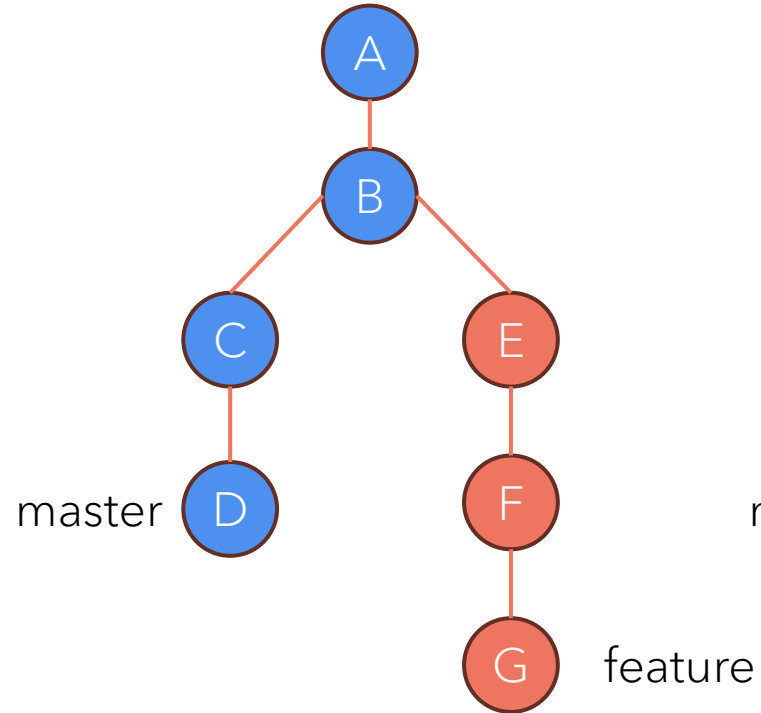
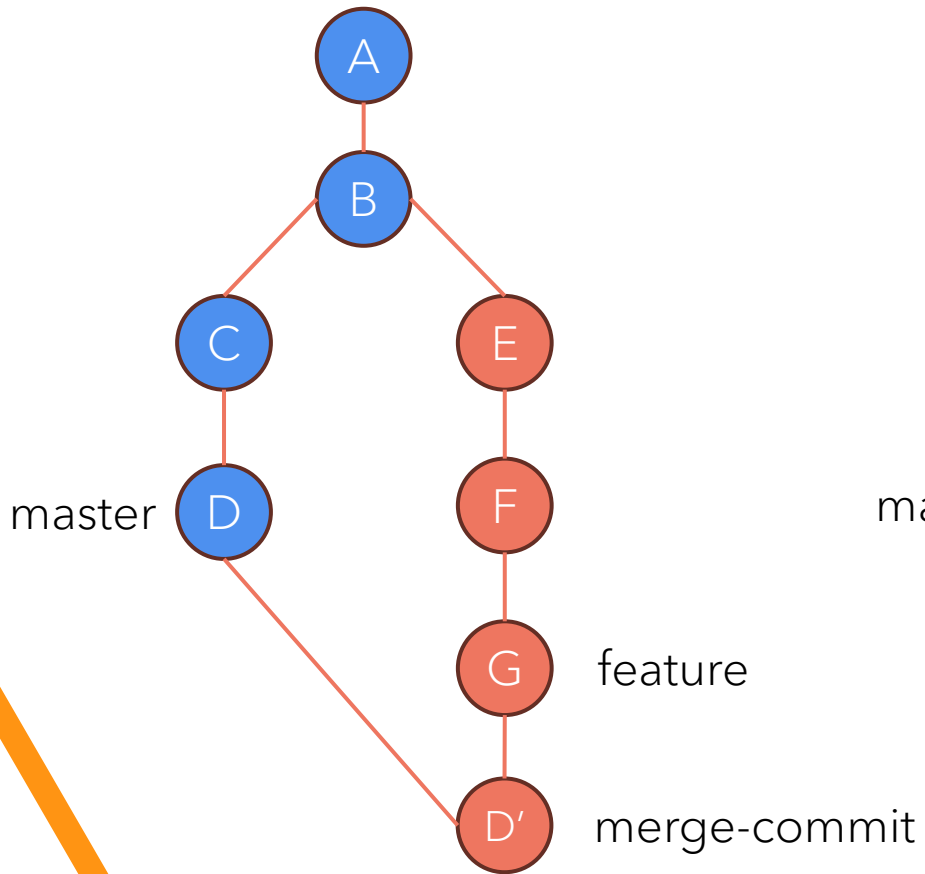
Git Rebase

- We have this git tree. We're developing new feature. After few day master have 2 new commit and we want to sync the master to our feature branch

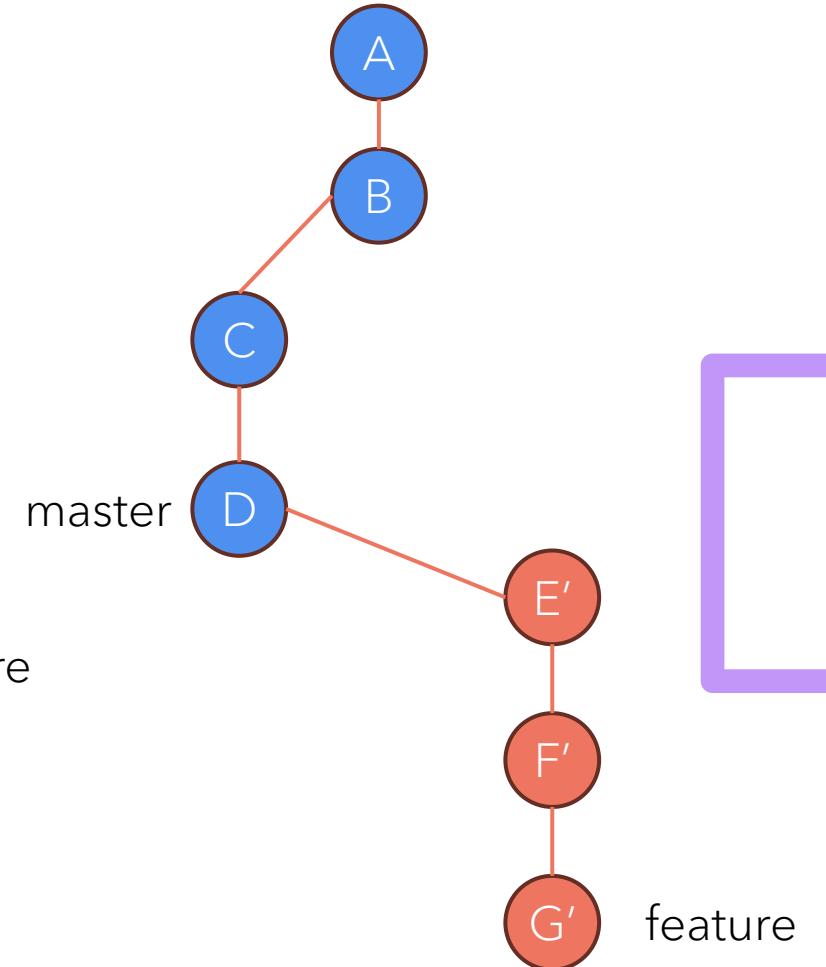


Git Rebase

git merge

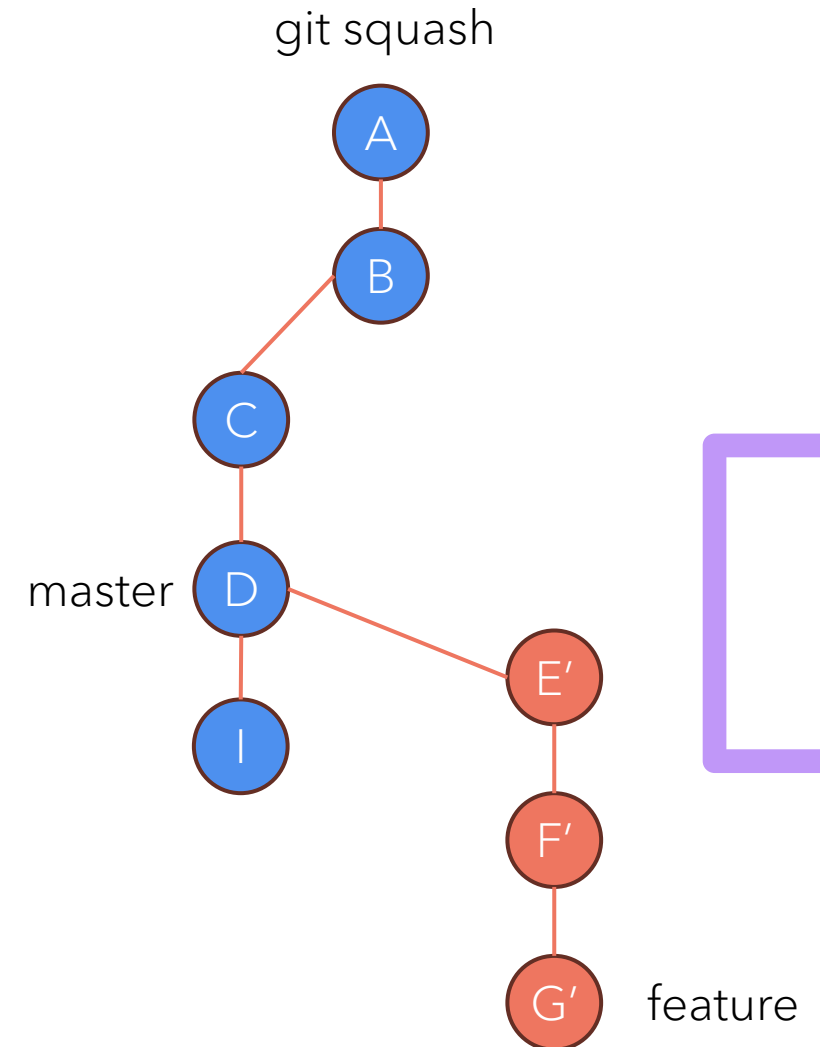


git squash



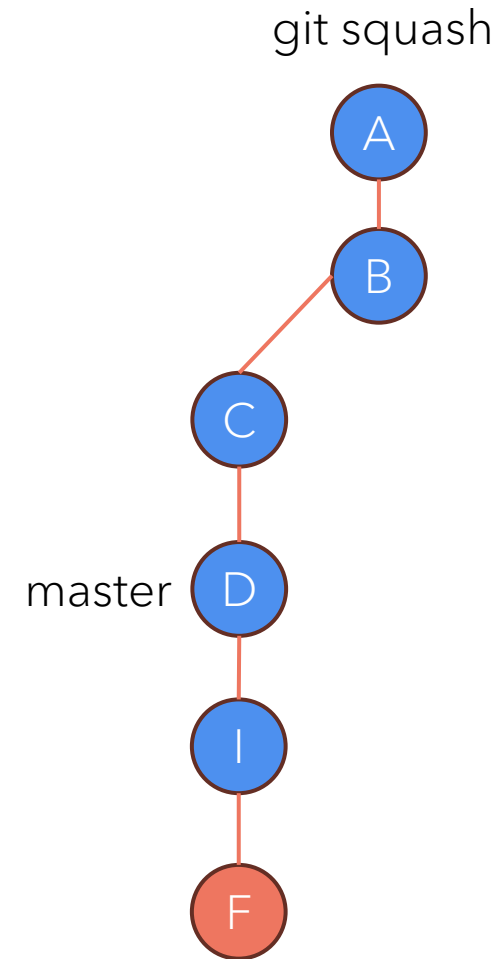
Git Rebase

- Before merge to master, we can squash the commit into one single commit. This will help history clear and avoid junk commit appear on the Git history



Git Rebase

- Before merge to master, we can squash the commit into one single commit. This will help history clear and avoid junk commit appear on the Git history (This could be important because if we have CI pipeline run on every commit, we don't want to run on all test or **un-related commit**)
- **Rebase rewrites history** → commits become *new* commits (E' and F',...).
- ⚠ Never **rebase commits that were already pushed to a shared branch** (other people may get conflicts).
- ✅ Safe use: rebasing your **local feature branch** before opening a Pull Request.



Other Git Command

- Stashing

```
# Save current work
git stash

# Apply stashed changes
git stash pop

# List stashes
git stash list
```

- Undoing Changes

```
git reset --soft HEAD~1

# Undo last commit (discard changes)
git reset --hard HEAD~1

# Create new commit that undoes changes
git revert <commit-hash>
```

.gitignore

A **.gitignore** file is a special text file in a Git repository that tells Git:
👉 which files or directories to ignore (not track) when committing changes.

That means, anything listed in **.gitignore** will not be added to version control

📁 Example .gitignore

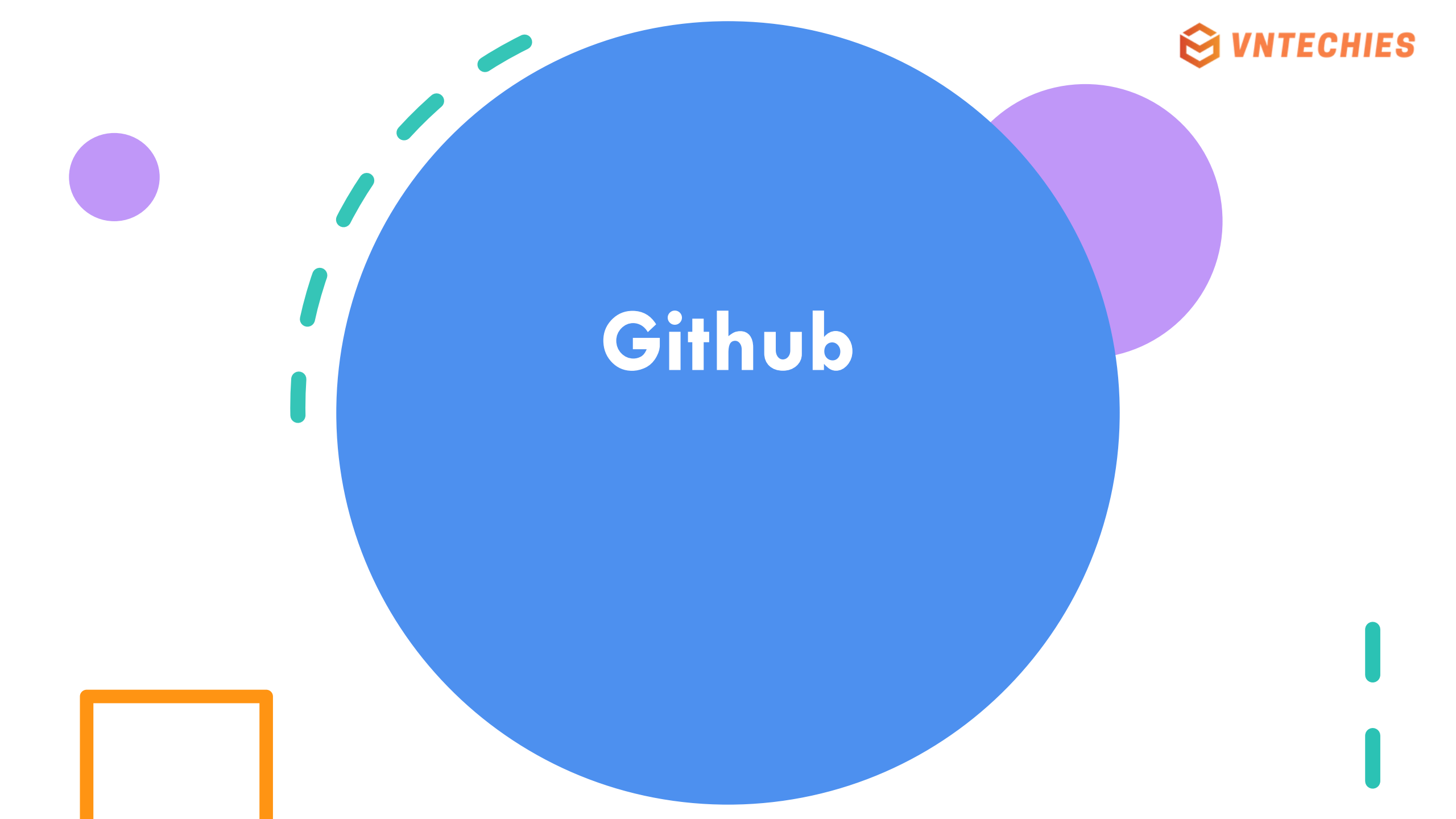
```
# Dependency directory
node_modules/

# Build output
dist/
build/

# Logs
*.log

# Environment variables
.env

# OS files
.DS_Store
Thumbs.db
```



Github

Github

👉 GitHub is a cloud-based platform built on top of Git (a version control system).

It allows developers to:

Host Git repositories online 📁

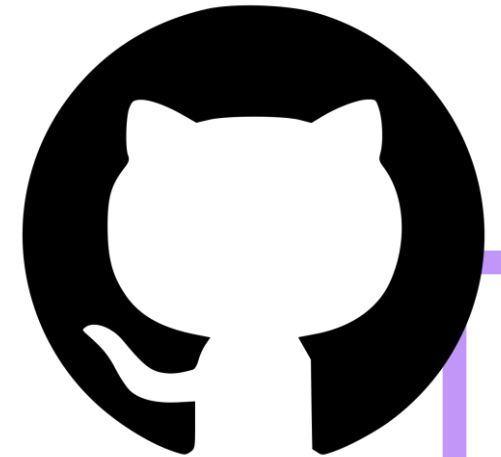
Collaborate with other developers 🤝

Manage projects with tools like pull requests, issues, and CI/CD ⚡

Think of GitHub as:

🔧 Git = The engine (local version control tool)

🏠 GitHub = The garage (a cloud hub to share, collaborate, and manage Git projects)



GitHub

Key Features:

- Remote repo hosting 🌐
- Pull Requests (collaboration) 🤝
- Issues (bug/feature tracking) 🐛
- GitHub Actions (automation/CI/CD) ⚡
- Project management 📁
- Security tools 🔒
- Social + Open source ecosystem 🌐





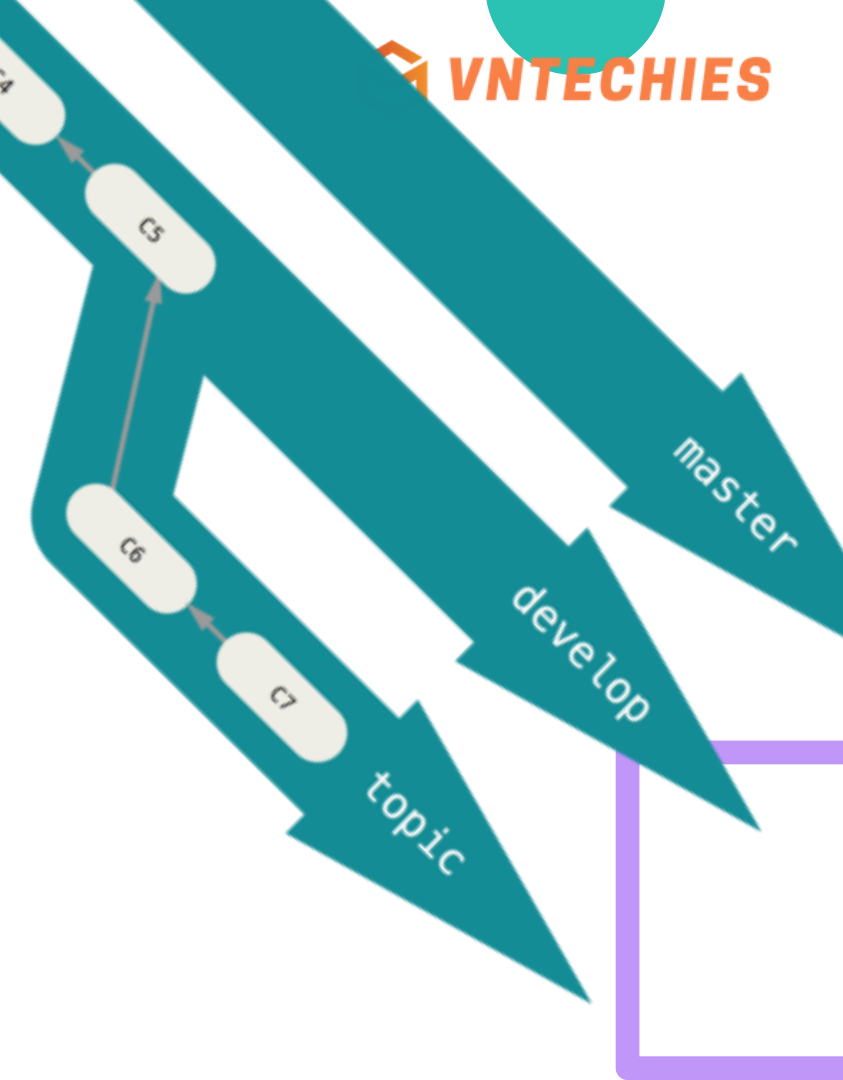
Git Branching Model

What is Git Branching model?

A Git branching model is a workflow or strategy that defines:

- How branches are named (e.g., main, develop, feature/login, etc.)
- How new work is branched out (features, bugfixes, hotfixes).
- How and when branches are merged back into stable branches.

It's basically a set of rules and guidelines for collaboration in Git so code development is organized, scalable, and manageable in a team.



Common Git Branching Models

Git Flow (most popular in teams)

- Two main branches that always exist:
 - **main** → Always production-ready code.
 - **develop** → Source branch for ongoing development.
- Supporting branches:
 - **feature/*** → New features (branch off **develop**).
 - **release/*** → Prepare a release (bug fixes & testing).
 - **hotfix/*** → Urgent fixes (branch off **main**).
- After completion:
 - Features → merged into **develop**.
 - Releases → merged into both **main** and **develop**.
 - Hotfixes → merged into both **main** and **develop**.



Common Git Branching Models

GitHub Flow

- Simple and lightweight (great for continuous deployment).
- Only one long-running branch:
 - **main** → Always deployable.
- Workflow:
 - Create a **feature** branch from **main**.
 - Work on feature/bug.
 - Open a Pull Request → review.
 - Merge back to **main**.
 - Deploy immediately.
- 📌 Used by: Startups, small teams, and projects requiring rapid deployment.



Common Git Branching Models

GitLab Flow

- Mix between GitHub Flow and Git Flow.
- Adds **environment branches**:
 - Example: `main` → production, pre-production, development, UAT.
- Developers branch off development, then deploy to staging/testing, and later to production.
- 📌 Used by: Enterprises with multiple environments (staging, testing, prod).



A large blue circle containing the text "Practice Lab". To its left is a small purple circle. To its right is a medium purple circle. Several teal dashed lines are scattered around the blue circle. In the bottom left corner is an orange square outline. In the bottom right corner are two teal vertical bars.

Practice Lab